

Practical Security, Governance, Migration, Validation, Rollback, and Trade-offs

Plain-language strategy package

Purpose and recommendation

This document explains in plain language how to move repositories and pipelines from five Azure DevOps organizations to one GitHub Enterprise environment while keeping Azure Boards in the chosen ADO organization. The scenario describes about 1,000 repositories and 1,500 pipelines, with varied DevOps maturity.

Recommendation: re-scope the program rather than stop everything or accelerate every migration. Continue repository moves that are low risk and well understood. Put more delivery capacity into pipeline conversion, developer support, and legacy runner solutions. Apply essential security protections immediately, then expand mandatory controls as the standard workflows prove reliable. Do not use disputed DORA metrics to rank teams or people.

This is a proposed strategy, not evidence of an enterprise implementation. Validate decisions against the actual inventory, security policies, licensing, and workload requirements.

1. Target platform in plain terms

- **GitHub Enterprise** becomes the central place for source code, pull requests, and the standard CI/CD approach.
- **Azure Boards** remains the work tracking system in one selected ADO organization. Validate links among work items, branches, commits, and pull requests before migration waves depend on them.
- **Reusable workflows and composite actions** are shared building blocks. Product teams use the same reviewed build and security steps rather than each team copying different pipeline logic.
- **Standard workloads** use managed/hosted runners where they meet the build needs. Legacy products that require x86, native tools, or a particular Windows environment use a separately managed, restricted runner pool.
- **Microsoft Fabric** stores analytics through Bronze, Silver, and Gold layers: Bronze keeps authorized source extracts, Silver cleans and standardizes them, and Gold contains agreed measures ready for reporting. Power BI presents the approved measures and their data-quality status.

The goal is not to centralize every decision. The central platform team provides safe, supported paths; service teams remain responsible for their code, tests, service ownership, and correct deployment and incident information.

2. Security controls

Minimum controls for every repository

- Require company identity controls such as single sign-on and managed account provisioning where available.
- Grant only the access needed for a person or automation account to do its job. Review access when owners or roles change.
- Protect important branches: require review, required checks, and ownership approval for sensitive files. Prevent direct changes that bypass agreed review.
- Scan for exposed credentials and vulnerable dependencies. Provide a clear route to fix findings and escalate critical issues.
- Keep workflow permissions read-only unless a step demonstrably needs more. Avoid storing long-lived credentials in source or logs; prefer short-lived federated credentials when supported.

- Allow only reviewed workflow actions. Pin external actions to approved immutable versions according to company policy, and review updates deliberately.
- Keep audit records and build artifacts for the period required by policy. Protect release environments with approvals and appropriate separation of duties.
- Isolate self-hosted runners from sensitive networks and from one another. Prefer disposable/ephemeral runners; restrict outbound access and do not reuse a runner with untrusted pull-request code.

Roll out controls without creating unsafe workarounds

Security should define the minimum protections and risk tiers with engineering, not simply switch on every rule at once. First measure whether required checks are available and stable. Provide a fast, visible exception process. If checks are noisy or routinely unavailable, fix them before making them universal. Track bypasses and blocked pull requests as signals to investigate, not just as failures to punish.

Exceptions

An exception is temporary, visible, and owned. Record the repository or pipeline, business reason, risk, compensating controls, approver, expiry date, and planned fix. Review exceptions monthly. Expired exceptions return to the owner and approver for renewal or remediation; they should not become undocumented permanent standards.

3. Governance and operating model

Who owns what

- **Executive sponsor:** resolves cross-organization priorities, approves investment and risk tolerance, and chairs major go/no-go decisions.
- **Central platform product team:** owns the GitHub paved road, shared workflows, runner service, migration tools, documentation, support, platform availability, and roadmap.
- **Security and identity teams:** define baseline controls, identity patterns, action/runner policy, evidence needs, and exception approval rules.
- **Service/product teams:** own their repository metadata, tests, release/deployment definitions, incident attribution, service criticality, and migration acceptance.
- **Data product steward / Fabric team:** owns metric definitions, data contracts, lineage, quality checks, freshness, and access to reported data.
- **Azure Boards owner:** maintains the selected ADO organization and validates work-item integration with GitHub.

Platform as an internal product

Treat the platform as a product used by engineering teams. Give it a funded roadmap, named product and technical owners, a support channel, clear service levels, changelogs, onboarding material, and adoption measures. Make the recommended path easier than building a custom one. Ask teams for feedback and measure time to first successful build and time to feedback, not only the number of repositories moved.

Decision and change governance

Shared workflows and security policy need versioning, peer review, release notes, and a tested upgrade path. Use a small pilot group before major changes. Publish who approves policy, who handles outages, how emergency changes are recorded, and how teams request exceptions. Keep ownership and escalation paths in the repository catalog.

4. Migration plan and gates

Start with an inventory

Before scheduling a move, capture a named owner, business criticality, repository and pipeline links, language/runtime, build tools, external dependencies, secrets and permissions, deployment destinations, artifact consumers, runner/architecture needs, test coverage, release frequency, and rollback method. Identify pipelines that share templates or release artifacts. Confirm what the source inventory actually contains; do not infer ownership or production status from names alone.

Use waves, not one large cutover

- **Wave 0 — discover and prepare:** select pilots, agree security baseline, test GitHub–Boards linking, record current build/deployment behavior, identify rollback and artifact retention needs.
- **Wave 1 — simple workloads:** move low-risk repositories with standard SDKs and few dependencies. Run old and new validation together for a defined period; compare results.
- **Wave 2 — common patterns:** migrate similar build pipelines in service-aligned batches using reusable workflows. Improve templates from pilot feedback before broad use.
- **Wave 3 — exceptions:** move native, x86, regulated, or tightly coupled workloads only after the required isolated runner and controls are ready. Approve time-bound exceptions where needed.
- **Wave 4 — retire old paths:** disable or remove an old pipeline only when the new path passes agreed checks, artifacts and permissions are accounted for, owners approve, and rollback has been rehearsed.

Batch by business service and dependency, not merely by repository count. Keep Azure Boards migration out of scope unless leadership explicitly changes that decision; validate integration links as part of repository and pipeline migration.

Cutover readiness checklist

A repository/pipeline is ready only when: its owner is known; expected build and test behavior is documented; secrets and permissions are reviewed; the new workflow passes representative cases; artifacts are available to their consumers; security controls are enabled or an approved exception exists; support and monitoring are in place; and a tested rollback plan names the decision-maker.

5. Validation and evidence

Validation should prove behavior, not just that a workflow file exists.

- **Static review:** check YAML syntax, workflow permissions, action versions, secret exposure, runner labels, and paths.
- **Build parity:** compare old and new build configuration, architecture, toolchain, and produced outputs. The supplied pipeline explicitly targets Windows/x86 and the v143 toolset; do not silently switch to Linux/x64 until application owners verify compatibility.
- **Test and coverage:** run the same relevant test suites and compare results. Publish coverage in a consistent format. Investigate missing tests rather than treating a successful compile as full validation.
- **Artifact comparison:** check artifact names, contents, retention, signing/provenance requirements, and downstream consumers.
- **Security checks:** validate branch rules, least-privilege token permissions, secret scanning, dependency checks, environment approvals, and runner isolation.
- **Operational rehearsal:** test a failed build, a runner failure, an urgent fix, and rollback. Confirm that responders know where logs and artifacts are.
- **Business acceptance:** obtain the service owner's explicit approval before retiring the old path.

Store evidence with the change: workflow run, build/test result, coverage output, artifact reference, approvals, known differences, exception record (if any), and rollback confirmation. A PR green check alone does not prove production deployment safety.

6. Rollback approach

Rollback means restoring a known-good way to build or release without losing source changes, artifacts, or audit history.

- Keep the old pipeline available but non-authoritative during the agreed dual-run period. Avoid allowing two pipelines to deploy to production at the same time.
- Keep source repository history and required release artifacts available according to retention policy.
- Use a clear cutover switch or documented procedure so the service owner can return to the old pipeline if the new one blocks a critical release or produces incorrect artifacts.
- Define triggers in advance: repeated unexplained build/test failures, missing or incorrect artifacts, security control failure, inability to restore, or unacceptable developer impact.
- Name the person who can call rollback, the team that executes it, and the communication channel. Record the event and perform a follow-up review.
- After rollback, contain the issue, preserve logs/evidence, fix the cause, and repeat validation before another cutover attempt.

Rollback should not be a reason to keep duplicate production deployment paths indefinitely. Set an end date and exit criteria for dual-run.

7. Trusted analytics and DORA metrics

Define metrics before building executive targets. For each measure specify the service and environment grain, time zone, event source, denominator, deduplication key, exclusions, attribution rules, freshness expectation, owner, and how missing data is shown.

- **Deployment frequency:** successful production deployments per service and time period. A build is not automatically a production deployment.
- **Lead time for changes:** elapsed time from an agreed change event (for example, first commit included in a release) to successful production deployment. Requires trustworthy mapping from deployed changes to commits.
- **Change failure rate:** production deployments that cause a qualifying failure divided by eligible production deployments, using an agreed incident attribution window and deduplication rule.
- **Mean time to restore:** elapsed time from service impact start to restoration for qualifying incidents; show event counts and unresolved incidents as context.

For the supplied workbook, profiling indicates 164 commits, 350 build records, 11 incidents, and two projects. The commit sheet has a `build\_sk` field while the build sheet has `build\_id`; verify the relationship rather than assuming the keys match. Build records do not show an explicit success/failure result or clearly identify production deployments. Three incidents are marked deployment-caused, and one incident's recorded resolution is earlier than its start. These limitations mean the workbook does not by itself support a defensible full DORA baseline. Validate source semantics with system owners, quarantine invalid intervals, publish source coverage and quality flags, and do not rank teams or people from incomplete data.

8. Key trade-offs and what to defer

- **Temporary dual-run costs time and compute**, but reduces the risk of losing a release path during migration. Keep it time-limited with explicit exit criteria.
- **A shared paved road improves consistency**, but can become a bottleneck. Let teams contribute through review and versioned extensions while maintaining centrally reviewed security requirements.
- **Hosted runners are simpler for standard builds**, while dedicated self-hosted Windows/x86 capacity may be necessary for legacy tools. Isolate and fund that exception path rather than weakening security for all workflows.
- **Strong branch rules improve assurance**, but noisy or unavailable checks can slow delivery. Stage enforcement after reliability and support are proven; keep emergency bypasses audited.

- **Central metric definitions improve consistency**, but teams may have different deployment models. Define common core rules and document service-specific exceptions openly.
- **More automation can reduce toil**, but AI-generated changes can be wrong or expose sensitive information. Start with approved, low-risk assistance and human review; do not initially permit autonomous merge or production deployment.

Defer mass rewrites, forced Azure Boards migration, blanket mandatory controls on unsupported workflows, broad self-hosted runner deployment, team-level DORA targets, and autonomous AI agents until evidence and ownership justify them.

9. Twelve-month roadmap

- **Months 1–3:** inventory owners and pipeline types; agree security and metric contracts; pilot standard and exception paths; record developer friction and baseline platform measures.
- **Months 4–6:** scale reusable workflows and migration support; establish isolated runner capability; validate GitHub–Boards linkage; implement initial Fabric Bronze/Silver/Gold flow and quality reporting.
- **Months 7–9:** migrate service-aligned waves; retire old paths only after gates pass; expand metric coverage, lineage, and supported security enforcement.
- **Months 10–12:** address remaining exceptions with business decisions; optimize reliability and cost; audit access, evidence, and metric quality; decide next-year improvements from outcomes.

10. First 90 days and decision gates

Days 0–30: appoint sponsor and owners; define inventory fields and risk tiers; identify x86/native needs; document existing release outputs and rollback; interview developers; agree metric definitions and source gaps.

Days 31–60: run representative pilots across standard and exception workloads; build reusable workflow patterns; compare old/new results; implement three executable data-quality checks; review runner security; run weekly feedback sessions.

Days 61–90: expand only low-risk workloads that pass readiness gates; apply branch protections to validated workflows; resolve priority runner exceptions; issue a provisional dashboard with coverage caveats; decide next wave using evidence.

At each decision gate, leadership should review migration throughput, build/test reliability, rollback readiness, security findings and exceptions, developer friction, runner cost, and analytics quality. If a gate fails, pause the affected workload or wave, not automatically the entire program.

11. Investment and success measures

Prioritize a small platform product team, migration automation and support, Windows/x86 runner isolation, identity/security engineering, Fabric/data stewardship, and developer enablement. Track owner coverage, successful migrations, escaped defects and rollback rate, build success/time-to-feedback, reuse of shared workflows, runner availability/cost, unresolved critical vulnerabilities, exception age, developer satisfaction, and DORA source coverage/freshness/quality.

Success is not simply “all repositories moved.” It is a supported, secure path that teams can use, with trustworthy evidence that the new pipeline behaves as expected.

Assumptions and implementation boundary

The organization size and target direction are taken from the candidate brief. This document describes a proposed operating model and strategy. The included sample code and workflow are examples and have not been run in the user's GitHub organization. The Fabric notebook is a starting-point sketch and has not been deployed to a live Fabric workspace. The workbook contains personal identifiers; do not publish it or its raw rows. The repository's sample data should remain clearly labeled synthetic unless approved, sanitized data is explicitly authorized.